PedidoPecasController

- repository: PedidoPecasRepository

+ criar(dto: PedidoPecasDTO): ResponseEntity<PedidoPecasDTO>
+ listar(): ResponseEntity<List<PedidoPecasDTO>>
+ buscarPorId(id: Long): ResponseEntity<PedidoPecasDTO>
+ adicionarItem(id: Long, dto: ItemPedidoDTO): ResponseEntity<PedidoPecasDTO>
+ atualizarStatus(id: Long, dto: AtualizarStatusDTO): ResponseEntity<PedidoPecasDTO>

PedidoPecasRepository

- instance: PedidoPecasRepository
- tabelaPedidos: List<PedidoPecas>
- contadorId: Long

- PedidoPecasRepository()
+ getInstance(): PedidoPecasRepository
+ salvar(pedidoPecas: PedidoPecas): PedidoPecas
+ listarTodos(): List<PedidoPecas>
+ buscarPorId(id: Long): Optional<PedidoPecas>
+ atualizar(id: Long, pedidoPecas: PedidoPecas): Optional<PedidoPecas>
+ deletar(id: Long): boolean

1 — 0..*

PedidoPecas

- codigoUnico: Long
- fornecedor: String
- status: StatusPedido
- itens: List<ItemPedido>
- dataCriacao: LocalDateTime
- dataUltimaAtualizacao: LocalDateTime

+ PedidoPecas(fornecedor: String)
+ adicionarItem(item: ItemPedido): void
+ calcularValorTotal(): Double
+ getCodigoUnico(): Long
+ setCodigoUnico(codigoUnico: Long): void
+ getFornecedor(): String
+ setFornecedor(fornecedor: String): void
+ getStatus(): StatusPedido
+ setStatus(status: StatusPedido): void
+ getItens(): List<ItemPedido>
+ setItens(itens: List<ItemPedido>): void
+ getDataCriacao(): LocalDateTime
+ setDataCriacao(dataCriacao: LocalDateTime): void
+ getDataUltimaAtualizacao(): LocalDateTime
+ setDataUltimaAtualizacao(dataUltimaAtualizacao: LocalDateTime): void

1 — 1..*

ItemPedido

- peca: Peca
- quantidade: Integer
- precoUnitario: Double

+ ItemPedido(peca: Peca, quantidade: Integer)
+ calcularSubtotal(): Double
+ getPeca(): Peca
+ setPeca(peca: Peca): void
+ getQuantidade(): Integer
+ setQuantidade(quantidade: Integer): void
+ getPrecoUnitario(): Double
+ setPrecoUnitario(precoUnitario: Double): void

0..*

PedidoPecasDTO

- codigoUnico: Long
- fornecedor: String
- status: String
- itens: List<ItemPedidoDTO>
- valorTotal: Double

PedidoPecasMapper

+ toEntity(dto: PedidoPecasDTO): PedidoPecas
+ toDTO(pedidoPecas: PedidoPecas): PedidoPecasDTO
+ toItem(dto: ItemPedidoDTO, peca: Peca): ItemPedido

«enumeration»
StatusPedido

ORCANDO
PENDENTE_PAGAMENTO
PAGO_FATURADO
ENTREGUE

ItemPedidoDTO

- codigoPeca: Long
- quantidade: Integer

PecaController

- repository: PecaRepository

+ cadastrar(dto: PecaDTO): ResponseEntity<PecaDTO>
+ listar(): ResponseEntity<List<PecaDTO>>
+ buscarPorId(id: Long): ResponseEntity<PecaDTO>
+ atualizar(id: Long, dto: PecaDTO): ResponseEntity<PecaDTO>
+ deletar(id: Long): ResponseEntity<Void>

PecaRepository

- instance: PecaRepository
- tabelaPecas: List<Peca>
- contadorId: Long

- PecaRepository()
+ getInstance(): PecaRepository
+ salvar(peca: Peca): Peca
+ listarTodos(): List<Peca>
+ buscarPorId(id: Long): Optional<Peca>
+ atualizar(id: Long, peca: Peca): Optional<Peca>
+ deletar(id: Long): boolean

1 — 0..*

Peca

- codigoUnico: Long
- nome: String
- codigoBarras: String
- fornecedorMarca: String
- quantidadeEstoque: Integer
- precoCusto: Double
- precoVenda: Double
- dataCadastro: LocalDateTime
- dataUltimaAtualizacao: LocalDateTime
- categoria: CategoriaPeca
- tamanho: String
- cor: String

+ Peca(nome: String, codigoBarras: String, fornecedorMarca: String, quantidadeEstoque: Integer, precoCusto: Double, precoVenda: Double, categoria: CategoriaPeca)
+ getCodigoUnico(): Long
+ setCodigoUnico(codigoUnico: Long): void
+ getNome(): String
+ setNome(nome: String): void
+ getCodigoBarras(): String
+ setCodigoBarras(codigoBarras: String): void
+ getFornecedorMarca(): String
+ setFornecedorMarca(fornecedorMarca: String): void
+ getQuantidadeEstoque(): Integer
+ setQuantidadeEstoque(quantidadeEstoque: Integer): void
+ getPrecoCusto(): Double
+ setPrecoCusto(precoCusto: Double): void
+ getPrecoVenda(): Double
+ setPrecoVenda(precoVenda: Double): void
+ getDataCadastro(): LocalDateTime
+ setDataCadastro(dataCadastro: LocalDateTime): void
+ getDataUltimaAtualizacao(): LocalDateTime
+ setDataUltimaAtualizacao(dataUltimaAtualizacao: LocalDateTime): void
+ getCategoria(): CategoriaPeca
+ setCategoria(categoria: CategoriaPeca): void
+ getTamanho(): String
+ setTamanho(tamanho: String): void
+ getCor(): String
+ setCor(cor: String): void

1

PecaDTO

- codigoUnico: Long
- nome: String
- codigoBarras: String
- fornecedorMarca: String
- quantidadeEstoque: Integer
- precoCusto: Double
- precoVenda: Double
- categoria: String
- tamanho: String
- cor: String

PecaMapper

+ toEntity(dto: PecaDTO): Peca
+ toDTO(peca: Peca): PecaDTO

«enumeration»
CategoriaPeca

MOTOR
SUSPENSAO
FREIOS
ELETRICA
ACESSORIOS

ServicoController

- repository: ServicoRepository

+ cadastrar(dto: ServicoDTO): ResponseEntity<ServicoDTO>
+ listar(): ResponseEntity<List<ServicoDTO>>
+ buscarPorId(id: Long): ResponseEntity<ServicoDTO>
+ atualizar(id: Long, dto: ServicoDTO): ResponseEntity<ServicoDTO>
+ deletar(id: Long): ResponseEntity<Void>

ServicoRepository

- instance: ServicoRepository
- tabelaServicos: List<Servico>
- contadorId: Long

- ServicoRepository()
+ getInstance(): ServicoRepository
+ salvar(servico: Servico): Servico
+ listarTodos(): List<Servico>
+ buscarPorId(id: Long): Optional<Servico>
+ atualizar(id: Long, servico: Servico): Optional<Servico>
+ deletar(id: Long): boolean

1 — 0..*

Servico

- codigoUnico: Long
- nomeServico: String
- tempoEstimadoMinutos: Integer
- custoTabelado: Double
- dataCriacao: LocalDateTime
- dataUltimaAtualizacao: LocalDateTime

+ Servico(nomeServico: String, tempoEstimadoMinutos: Integer, custoTabelado: Double)
+ getCodigoUnico(): Long
+ setCodigoUnico(codigoUnico: Long): void
+ getNomeServico(): String
+ setNomeServico(nomeServico: String): void
+ getTempoEstimadoMinutos(): Integer
+ setTempoEstimadoMinutos(tempoEstimadoMinutos: Integer): void
+ getCustoTabelado(): Double
+ setCustoTabelado(custoTabelado: Double): void
+ getDataCriacao(): LocalDateTime
+ setDataCriacao(dataCriacao: LocalDateTime): void
+ getDataUltimaAtualizacao(): LocalDateTime
+ setDataUltimaAtualizacao(dataUltimaAtualizacao: LocalDateTime): void

0..*

ServicoDTO

- codigoUnico: Long
- nomeServico: String
- tempoEstimadoMinutos: Integer
- custoTabelado: Double

ServicoMapper

+ toEntity(dto: ServicoDTO): Servico
+ toDTO(servico: Servico): ServicoDTO

OrdemServicoController

- repository: OrdemServicoRepository
- servicoRepository: ServicoRepository

+ criar(dto: OrdemServicoDTO): ResponseEntity<OrdemServicoDTO>
+ listar(): ResponseEntity<List<OrdemServicoDTO>>
+ buscarPorId(id: Long): ResponseEntity<OrdemServicoDTO>
+ atualizarStatus(id: Long, dto: AtualizarStatusDTO): ResponseEntity<OrdemServicoDTO>

OrdemServicoRepository

- instance: OrdemServicoRepository
- tabelaOrdens: List<OrdemServico>
- contadorId: Long

- OrdemServicoRepository()
+ getInstance(): OrdemServicoRepository
+ salvar(ordemServico: OrdemServico): OrdemServico
+ listarTodos(): List<OrdemServico>
+ buscarPorId(id: Long): Optional<OrdemServico>
+ atualizar(id: Long, ordemServico: OrdemServico): Optional<OrdemServico>
+ deletar(id: Long): boolean

1 — 0..*

OrdemServico

- codigoUnico: Long
- nomeCliente: String
- placaVeiculo: String
- descricaoProblema: String
- status: StatusOrdemServico
- servicos: List<Servico>
- valorTotal: Double
- dataAbertura: LocalDateTime
- dataUltimaAtualizacao: LocalDateTime

- OrdemServico(builder: Builder)
+ builder(): Builder
+ getCodigoUnico(): Long
+ getNomeCliente(): String
+ getPlacaVeiculo(): String
+ getDescricaoProblema(): String
+ getStatus(): StatusOrdemServico
+ getServicos(): List<Servico>
+ getValorTotal(): Double
+ getDataAbertura(): LocalDateTime
+ getDataUltimaAtualizacao(): LocalDateTime
+ setCodigoUnico(codigoUnico: Long): void
+ setStatus(status: StatusOrdemServico): void
+ calcularValorTotal(): Double

0..*

OrdemServico.Builder

- nomeCliente: String
- placaVeiculo: String
- descricaoProblema: String
- status: StatusOrdemServico
- servicos: List<Servico>
- dataAbertura: LocalDateTime

+ nomeCliente(nomeCliente: String): Builder
+ placaVeiculo(placaVeiculo: String): Builder
+ descricaoProblema(descricaoProblema: String): Builder
+ status(status: StatusOrdemServico): Builder
+ servicos(servicos: List<Servico>): Builder
+ dataAbertura(dataAbertura: LocalDateTime): Builder
+ build(): OrdemServico

OrdemServicoDTO

- codigoUnico: Long
- nomeCliente: String
- placaVeiculo: String
- descricaoProblema: String
- status: String
- codigosServicos: List<Long>
- valorTotal: Double

OrdemServicoMapper

+ toEntity(dto: OrdemServicoDTO, servicos: List<Servico>): OrdemServico
+ toDTO(ordemServico: OrdemServico): OrdemServicoDTO

«enumeration»
StatusOrdemServico

ABERTO
PENDENTE_PAGAMENTO
PAGO
EM_EXECUCAO
EXECUTADO

AtualizarStatusDTO

- status: String